

Semester Project: Domain Discovery, Recommendation, and Graph Intelligence

Milestones: Dataset Charter, Feature Engineering, Dimensionality Reduction, and Prototype-Based Clustering

Iam Anthony Marcelo Alvarez Orellana
Jeffrey Ulises Diaz Villanueva
Paula Jimena Mancilla Cienfuegos
Fernando Samuel Paredes Espinoza

1 Project Proposal

- **Domain:** Global Digital Video Game Marketplace (Steam).
- **Problem Statement:** With over 80,000 titles available, the Steam platform faces an acute “**Choice Overload**” problem. Users typically gravitate toward high-budget (AAA) titles, while high-quality niche (Indie) games remain undiscovered. Traditional popularity-based ranking systems fail to capture latent connections between complex gameplay preferences and community-driven structures.
- **Expected Product Question:** How can we improve game discovery through a hybrid system that leverages community-driven tag centrality within a graph and historical user behavior (playtime) to recommend titles with high satisfaction probability?
- **Suitability for the course:** This dataset is ideal for building the four mandatory course layers:
 - **Catalog Layer:** Technical and commercial game metadata (genres, developers, tags).
 - **Feature Layer:** Vectorization of game descriptions and community tags for similarity analysis.
 - **Interaction Layer:** A dataset of ~480k unique interactions including playtime and voting behavior.
 - **Graph Layer:** Bipartite User-Game graphs and Tag-to-Tag co-occurrence networks.

2 Source Inventory

- **Source URLs:**
 - Steam Games Dataset 2025 - Artemiy Ermilov (Kaggle)
 - Steam Games Reviews 2024/2025 - Artemiy Ermilov (Kaggle)

- **Licenses:** CC0: Public Domain / Kaggle Dataset Terms of Use.
- **Raw File Formats:** CSV, multiple files for reviews, and a single CSV for the game catalog.
- **Estimated Size:** Approximately 128 million total records in the universe, sampled down for the V1 working subset.

3 Schema Draft

The system follows a star schema architecture where **AppID** serves as the central join key between the catalog and the interaction layers.

- **Main Entities:**
 - **Games (Catalog):** Technical, commercial, and descriptive metadata.
 - **Reviews (Interactions):** User sentiment, engagement metrics, and behavioral data.
- **Keys:**
 - **AppID:** Primary Key for Games table; Foreign Key for Reviews.
 - **author_steamid:** Unique identifier for users (Author).
 - **recommendationid:** Unique identifier for each interaction/review record.
- **Expected Joins:** Inner join on **AppID** to enrich interaction data with genres, tags, and developer information.

4 Data Dictionary Draft (Core Features)

Table: Games (Catalog)

Column	Type	Description
AppID	Integer	Unique identifier for the game on the Steam platform.
name	String	Official title of the videogame.
genres	String	Primary categories, for example, Action, RPG, Strategy.
tags	String	Community-driven descriptors used for graph construction.
price	Float	Current retail price in USD.
average_playtime_for_owners	Integer	Average playtime across all owners, used as a global baseline.

Table: Reviews (Interactions)

Column	Type	Description
<code>recommendationid</code>	Integer	Unique ID for the specific review record.
<code>author_steamid</code>	String	Anonymized unique ID of the user.
<code>voted_up</code>	Boolean	Target variable: indicates if the user recommends the game.
<code>author_playtime_forever</code>	Float	Total hours played by the specific user, used as an engagement metric.
<code>author_num_games_owned</code>	Integer	User profile feature representing library depth.

5 Scale Analysis (V1)

- **Total Raw Files Found:** 79,994 files.
- **V1 Working Subset:** 500 game files processed.
- **Rows in V1:** 480,025 unique interaction records.
- **Columns:** 67 original columns reduced to 12 prioritized features in the pipeline.
- **Sparsity Estimate:** ~99.8%, standard for large-scale recommendation domains.
- **Memory Footprint:** ~115 MB in optimized Parquet format.

6 Ethics and Access Note

- **Provenance:** Data is sourced from the public Steamworks API and structured by researchers for academic purposes.
- **Authorization:** Data use is permitted under the Steam API Terms of Service for research and non-commercial development.
- **Personal Data Risks:** The dataset includes `author_steamid`. While these are public profiles, there is a risk of profile tracking.
- **Risk Mitigation:** The ingestion pipeline excludes free-text comments to prevent PII leakage. IDs are treated as opaque numerical identifiers. No emails, real names, or financial data are collected or stored.

7 Technical Expectation: Ingestion Command

To reproduce the processed V1 dataset, run the following command from the project root:

```
# 1. Activate virtual environment
# Windows: .\venv\Scripts\activate
# Mac/Linux: source venv/bin/activate

# 2. Run ingestion pipeline
python src/ingest.py
```

8 Exploratory Data Analysis (EDA)

The EDA was generated from `data/processed/steam_v1.parquet`. Figures are exported as PNG files and included below.

Important methodological note. This report is based on the V1 subset, which contains 500 review files, not the full universe of available Steam review files. Therefore, absolute counts and ranking positions can vary across runs depending on which files are included in the subset and how source files are discovered in the file system. The plots should be interpreted as an initial analytical baseline for pattern discovery, not as final population-level estimates.

1) Recommendations Distribution

This chart summarizes the class balance of user sentiment, recommended vs not recommended. In our sample, positive recommendations dominate, which is a common behavior in review ecosystems. Even if exact counts change with a different subset, this figure is useful to assess target imbalance before modeling and to plan metrics beyond raw accuracy.

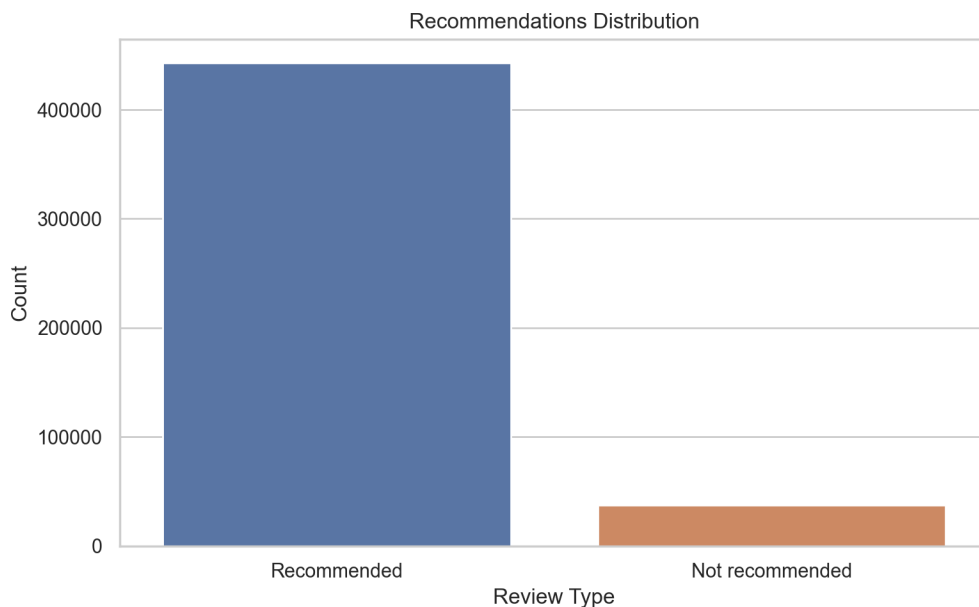


Figure 1: Distribution of recommendation labels: Recommended vs Not recommended.

2) Playtime Distribution

This histogram shows strong right skew: most users accumulate relatively low-to-moderate playtime, while a smaller group contributes very high values. The 99th-percentile cap improves readability and prevents outliers from compressing the main body of the distribution. The exact tail magnitude may vary with sample composition, but the skewed shape is typically stable.

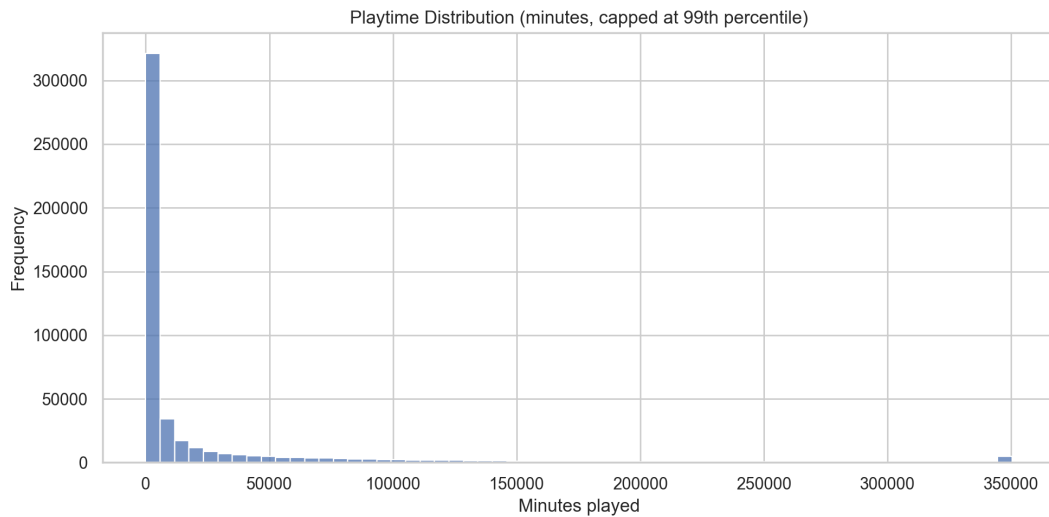


Figure 2: Playtime distribution in minutes, capped at the 99th percentile for readability.

3) Top Games by Review Volume

This ranking identifies which titles concentrate the largest number of interactions in the working subset. It is useful for understanding exposure concentration and potential popularity bias in downstream recommenders. Since this is subset-based, specific game order can shift between runs, but the long-tail structure usually remains.

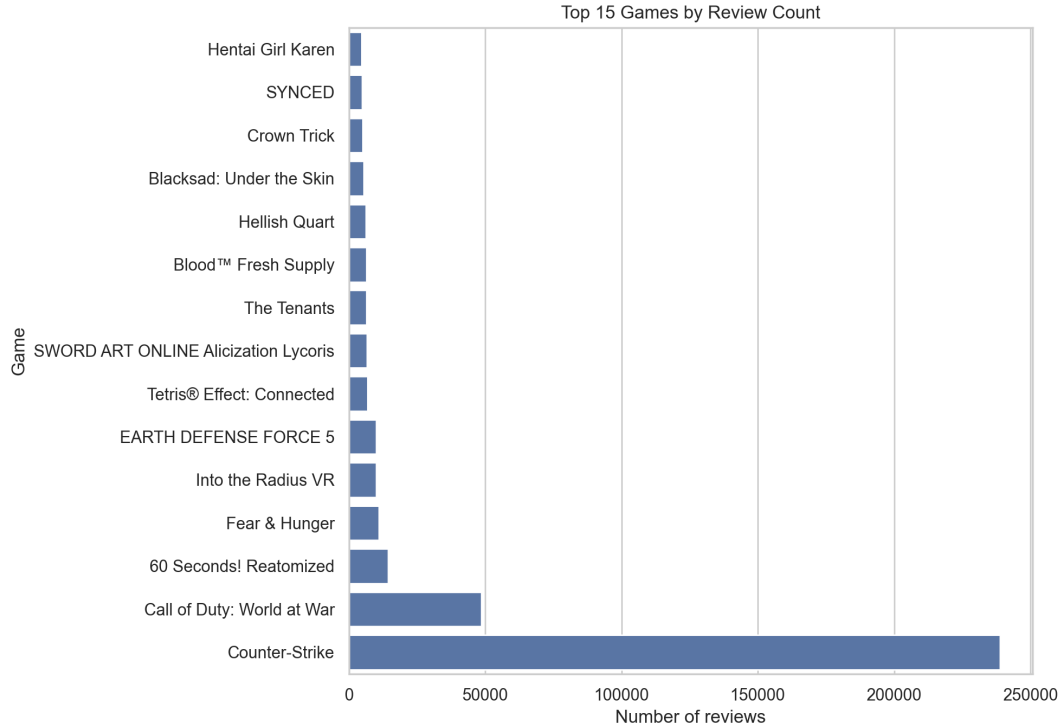


Figure 3: Top games ranked by number of review records in the V1 subset.

4) Recommendation Rate for Top Games

This figure compares satisfaction proxy, represented by recommendation rate, among the most-reviewed titles. It separates high-volume games that are broadly well received from those with mixed sentiment. Because this metric is conditional on sample coverage, values should be interpreted as preliminary and re-estimated on larger or full data before product decisions.

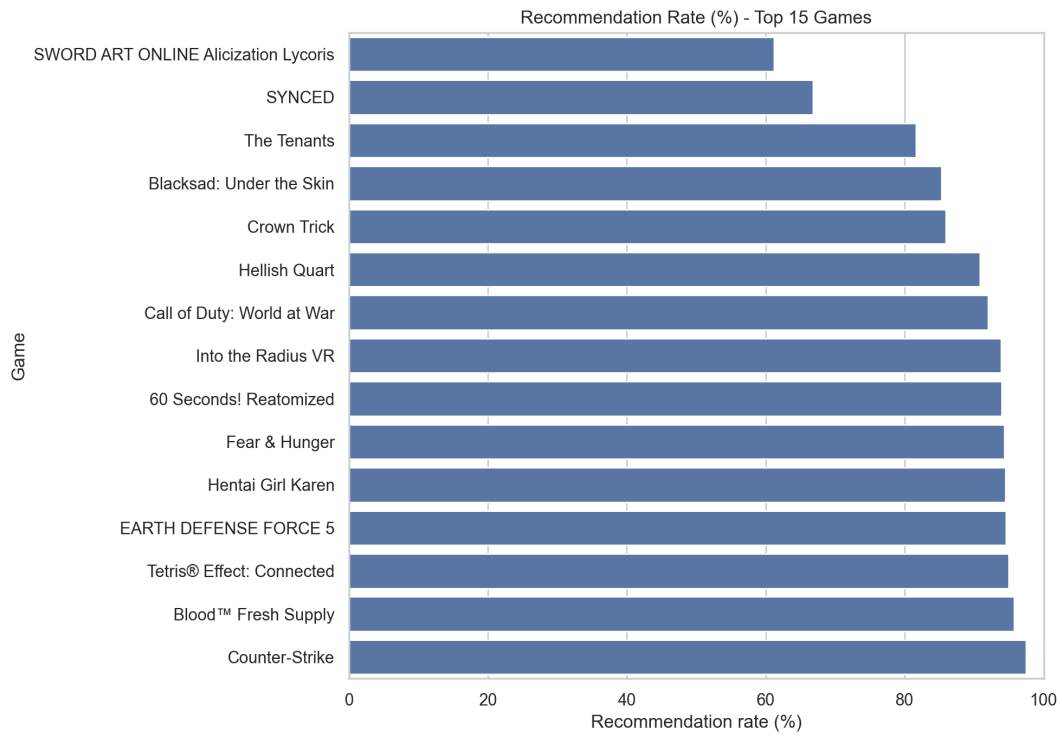


Figure 4: Recommendation rate (%) among games with the highest review volume.

5) Top Genres by Review Volume

This chart highlights which genres dominate observed interaction activity after joining review and catalog data. It helps prioritize segment-level modeling and feature engineering by genre density. Genre rankings may move as the subset changes, but they still provide an initial signal of market attention in the sampled data.

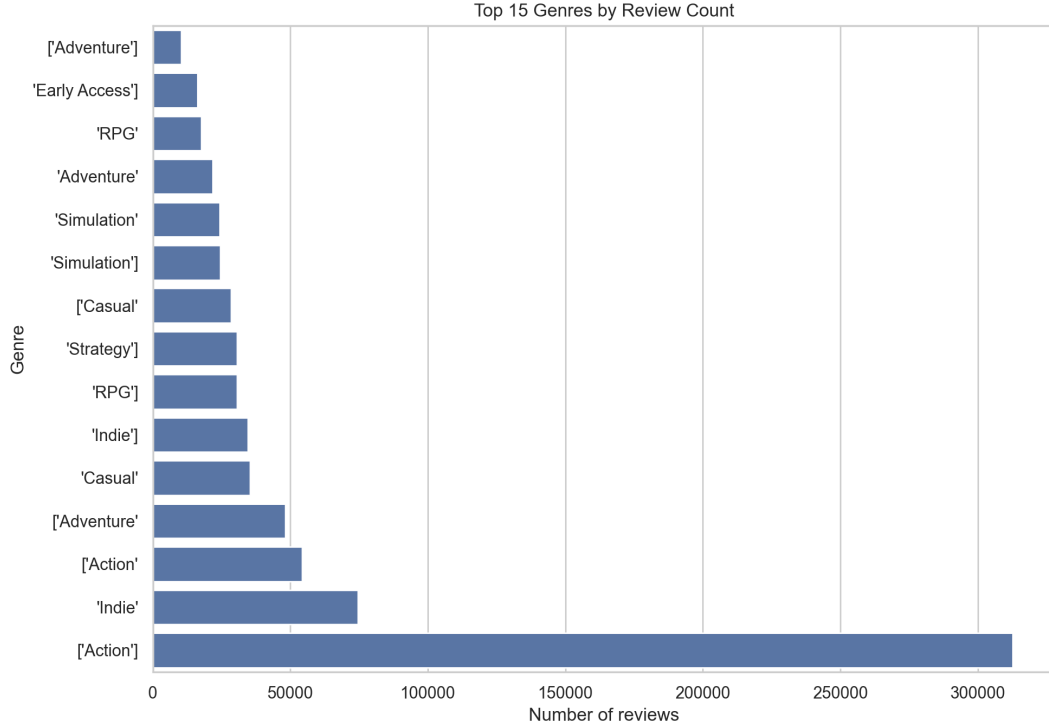


Figure 5: Most frequent genres in the interaction dataset after catalog join.

9 Feature Engineering and Representation (Week 5)

Once the dataset was curated, the next milestone was transforming the raw catalog into numerical representations that downstream algorithms (clustering, similarity search, graph projections) can consume. Three complementary feature blocks were built from `games_may2024_full.csv` (~88k games), each capturing a distinct dimension of a videogame.

Block	Matrix	Dimension	Type	Construction
Numerical + temporal	<code>X_numeric</code>	$88k \times 23$	dense	<code>StandardScaler</code> + 1/99 percentile clipping
Categorical (tags/genres)	<code>X_categorical</code>	$88k \times 5000$	sparse CSR	<code>MultiLabelBinarizer</code>
Text (descriptions)	<code>X_text</code>	$88k \times 5000$	sparse CSR	<code>TfidfVectorizer</code> (1–2 grams)

The three blocks live on incompatible scales (commercial magnitudes, binary indicators, term-frequency weights), so they cannot be naively concatenated. Each must be reduced and re-scaled before any unified representation is built.

PCA on `X_numeric`

Principal Component Analysis was applied to the dense numerical block. Ten components preserve approximately **90% of the total variance**, with a Frobenius reconstruction error

close to 0.10. A key preprocessing decision was clipping numerical values at the 1st and 99th percentiles: without it, AAA outliers dominated the first principal component entirely, masking the structure of mid-tier and indie titles.

TruncatedSVD on `x_categorical`

For the sparse tag matrix, PCA is unsuitable because it requires centering, which would densify the matrix and exhaust memory on an $88k \times 5000$ grid. `TruncatedSVD` operates directly on the sparse CSR representation. After filtering tags with frequency below 30, the 493 surviving dimensions were reduced to **50 latent components**, retaining ~62% of cumulative explained variance — a favorable trade-off between fidelity and compactness for clustering.

10 Prototype-Based Clustering with K-means (Week 6)

Motivation and Objective

Up to Week 5 we knew how to *represent* games as numerical vectors. A vector alone, however, does not tell us *which games are similar to each other*. Week 6 introduces the first unsupervised learning algorithm of the project: K-means, whose objective is to discover a partition $\{C_1, \dots, C_K\}$ minimizing the total within-cluster squared distortion to centroids.

Formal Setup and Lloyd’s Algorithm

Given n observations $x_i \in \mathbb{R}^d$ and K clusters with centroids μ_k , K-means minimizes

$$J(R, \mu) = \sum_{i=1}^n \sum_{k=1}^K r_{ik} \|x_i - \mu_k\|_2^2$$

where $r_{ik} \in \{0, 1\}$ is the assignment indicator. The problem is NP-hard in general; Lloyd’s algorithm tackles it with block-coordinate descent, alternating an **assignment step** (each point goes to its nearest centroid) and an **update step** (each centroid becomes the arithmetic mean of its assigned points). The mean is the optimal centroid *because* the loss is squared-Euclidean — with ℓ_1 loss the optimum would be the median, and with Bregman divergences yet another estimator.

Monotonic Descent and Non-convexity

Because each step minimizes J over one block while keeping the other fixed, $J^{(t+1)} \leq J^{(t)}$ for every t . Combined with $J \geq 0$, this guarantees convergence to a local minimum or stationary point. However, the joint problem in (R, μ) is *non-convex*: different initializations yield different local optima. In the Steam context, a bad initialization could collapse stylistically opposite titles such as *Dark Souls* and *Candy Crush* into the same cluster purely by random seeding. We mitigate this with `k-means++` for smart initialization and `n_init=10` to retain the best of ten independent runs.

Adaptation to the Steam Domain

The course laboratory clustered songs using dense audio vectors (`pachamix_audio_core`, 518D). Our domain differs sharply: Steam games are described primarily by binary tags, genres, and categories. The **key business decision** was to cluster on the **semantic space (tags)** rather than the commercial one (price, sales, reviews). The reason is product-driven: our goal is to surface Indie titles that drown in the 88k-game catalog. Clustering by price would place an excellent Indie next to a AAA simply because both cost USD 60 — useless for discovery. Clustering by *game type* places that Indie roguelike next to *Hades* or *Dead Cells*, which is precisely where players actually find it.

Week 6 Pipeline

The full clustering pipeline applies five processing stages, each with an explicit business or technical reason:

1. **Catalog filter:** the 87,806 raw games are reduced to 69,943 by dropping titles without any tag, since unlabeled games cannot be clustered semantically.
2. **Binarization:** `MultiLabelBinarizer` encodes genres, tags, and categories as binary columns. Developers, publishers, and language metadata are excluded as noise (a game does not resemble another just because both are localized in Chinese).
3. **TruncatedSVD to 50D:** the 493 surviving tags are compressed into 50 latent “themes”, working directly on the sparse representation.
4. **L2 normalization:** every game vector is projected onto the unit sphere S^{49} , so that highly tagged games do not weigh more than sparsely tagged ones.
5. **K-means:** the final partition is computed with `n_clusters=K, n_init=10, random_state=42`.

Theoretical Justification of SVD + L2

K-means with squared-Euclidean distance implicitly assumes that the *norm* of each vector encodes a relevant magnitude. In raw `X_categorical`, the norm of a row equals $\sqrt{\text{number of tags}}$, so K-means would cluster by *how many* tags a game has rather than *which kind* of tags. Normalizing each row by its ℓ_2 norm forces every game onto the unit sphere, where

$$\|x_i - x_j\|_2^2 = 2(1 - \langle x_i, x_j \rangle).$$

Squared-Euclidean distance on the sphere is a monotone transformation of cosine similarity — the correct geometric notion for binary tag indicators.

Ablation — Empirical Evidence for SVD + L2

To validate the SVD + L2 design choice empirically, we ran K-means on three variants of the same pipeline and measured the silhouette score:

Space	Silhouette	Largest cluster share
Raw sparse (binary tags)	0.001	22.7%
SVD only (50D dense)	0.036	21.9%
SVD + L2 (chosen)	0.069	25.3%

The L2 step alone produces an improvement of roughly **70× in silhouette** over the raw representation. In business terms, without L2 a heavily tagged title (30+ tags) would form its own cluster simply by virtue of being densely annotated; with L2 it is grouped by *what kind of game it actually is*. The choice is measured, not aesthetic.

Part A — Manual Lloyd’s Algorithm (Demonstration of the Descent)

To verify Lloyd’s monotonic-descent property by hand, a single iteration was run from five distinct random seeds on `PCA(X_numeric, 10)` with $K = 6$. For each seed we computed J before and after one assignment-plus-update step.

Seed	J before	J after	ΔJ	Descent OK
1	59,335.6	42,933.7	−16,402	Yes
2	59,448.2	47,519.6	−11,929	Yes
3	56,224.3	42,521.8	−13,702	Yes
4	51,373.5	39,969.5	−11,404	Yes
5	56,081.7	42,075.2	−14,007	Yes

All five seeds satisfy $J^{(1)} \leq J^{(0)}$, confirming Lloyd’s monotonicity empirically. For reference, `sklearn` run to full convergence with `n_init=10` reaches a final SSE of approximately 35,200 — consistent with the fact that further iterations continue to decrease J beyond the single hand-computed step.

Part B — Sweeping over K

K	Inertia	Silhouette	Min size	Max size
4	37,476	0.0643	12,655	21,324
6	35,319	0.0676	4,793	18,932
8	33,800	0.0655	4,649	13,220
10	32,621	0.0676	4,325	8,650

We chose $K = 6$ based on three criteria: highest silhouette in the bottom-knee region, no degenerate clusters, and direct interpretability against domain knowledge.

The Six Resulting Clusters

C	Size	Identity (top tags by lift)
C0	13,586 (19%)	Casual / Puzzle — match 3, logic, clicker, relaxing, hidden object
C1	18,742 (27%)	Action — action (96%), roguelike, bullet hell, platformer
C2	10,674 (15%)	Local multiplayer — shared screen, local co-op (lift 6×)
C3	13,145 (19%)	Narrative / JRPG — RPGMaker, visual novel, interactive fiction
C4	~6,000	VR — VR exclusive (88%), tracked controllers (84%)
C5	~7,800	Strategy — grand strategy, RTS, city builder, management

Cluster stability was evaluated via the Adjusted Rand Index (ARI) across five seeds, with mean off-diagonal ARI above 0.8 — indicating that the partition is consistent and not an artifact of initialization.

Known Failure Modes and Bridge to Week 7

K-means has structural limitations that surface in this domain:

- **Hybrid games at boundaries:** a “souls-like roguelike” is forced into C1 and its individual silhouette is close to zero.
- **Variable density:** cluster C1 (Action, ~19k games) is an amorphous mass of FPS, metroidvania, and bullet hell that K-means cannot subdivide without producing degenerate clusters.
- **Outliers:** asset flips with copied tags distort centroids; K-means has no rejection mechanism.
- **K is exogenous:** silhouette is a heuristic, not a proof. $K = 6$ is *defended*, not *proven*.

Week 7 addresses these limitations with density-based methods (DBSCAN / HDBSCAN), which support non-convex clusters and an explicit **noise** label, and hierarchical methods that produce a multi-scale dendrogram (action → roguelike → bullet hell).